

Java Enums

Constants with Power & Intelligence

codehive

Introduction to Enums

An **enum** (short for "enumeration") is a special "class" in Java that represents a group of **constants** (unchangeable variables). Unlike regular variables, enums define a fixed set of values that a variable can take.

Enums are ideal for data that doesn't change during the lifecycle of an application, such as:

- Days of the week
- Compass directions (NORTH, SOUTH, etc.)
- Application states (LOADING, SUCCESS, ERROR)
- Product levels (LOW, MEDIUM, HIGH)

```
enum Level {  
    LOW,  
    MEDIUM,  
    HIGH  
}
```

1. Working with Enums

You can define an enum inside or outside a class. To access its values, you use the dot syntax.

```
public class Main {  
    enum Level {  
        LOW,  
        MEDIUM,  
        HIGH  
    }  
  
    public static void main(String[] args) {  
        Level myVar = Level.MEDIUM;  
        System.out.println(myVar);  
    }  
}
```

```
> MEDIUM
```

Note: Enum constants are written in uppercase letters by convention because they are essentially final variables.

2. Enums in Switch Statements

Enums are frequently used with [switch statements](#) to check for corresponding values, making your code much cleaner than using multiple "if-else" blocks.

```
Level myVar = Level.MEDIUM;

switch(myVar) {
    case LOW:
        System.out.println("Low level");
        break;
    case MEDIUM:
        System.out.println("Medium level");
        break;
    case HIGH:
        System.out.println("High level");
        break;
}
```

```
> Medium level
```

3. Looping Through Enums

Java enums have a built-in static method called `values()`. This method returns an array containing all the constants of the enum, which is useful for iteration.

```
for (Level myVar : Level.values()) {  
    System.out.println(myVar);  
}
```

```
> LOW  
> MEDIUM  
> HIGH
```

Pro Tip: Use the `ordinal()` method to find the position (index) of an enum constant, starting from 0.

4. Advanced Features

Enums in Java are much more powerful than in other languages. They can have **attributes**, **constructors**, and **methods**.

```
enum Level {
    LOW("Low level"),
    MEDIUM("Medium level"),
    HIGH("High level");

    private String description;

    // Constructor (must be private)
    private Level(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}
```

This allows you to attach metadata to each constant directly.

5. Executing Advanced Enums

By combining the `values()` method with custom methods, you can build descriptive logic for your application constants.

```
public class Main {  
    public static void main(String[] args) {  
        // Loop through with description  
        for (Level myVar : Level.values()) {  
            System.out.println(myVar + ": " + myVar.getDescription());  
        }  
    }  
}
```

```
> LOW: Low level  
> MEDIUM: Medium level  
> HIGH: High level
```

Summary Checklist

- ✓ Enums represent a group of **fixed constants**.
- ✓ They make code more **readable** and **type-safe**.
- ✓ Enums can be used easily in **Switch** statements.
- ✓ The `values()` method allows easy iteration.
- ✓ Enums can have their own **fields** and **constructors**.

End of Module

codehive